BriteTest
Glossary Reference

Document Version: 1.0
Runner Version: 1.0.0
Test Version: 1.0.0

This glossary defines generally used terms in the documentation and terms often used in the testing domain. It is a companion document to the Documentation Guide.

Preface

This document is for users and contributors who need quick access to the definitions of terms or to browse through the terms.

For a list of other documents and the repository layout, see the Documentation Guide (`Documentation_Guide.md`).

A printer-friendly PDF file for this document is available in `docs/pdf/`.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the Runner API version current at the time this document version was published and is defined by its `RA_VERSION` macro.
- The **Test** column records the Test API version current at the time this document version was published and is defined by its `TA_VERSION` macro.
- Both Runner and Test use the version format `"M.m.p"` (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks released updates to this document and does not correspond to a minor or patch version. `u` increments when document changes are published in a release without a change to `M`, and it resets to `0` when `M` is incremented.

Table of Contents

1. Introduction

The glossary defines generally used terms in the documentation and terms often used in the testing domain.

Note: '†' indicates the definition of a term that has a specific meaning in the documention which may differ from the meaning of the term in the testing domain or other contexts.

Note: For specific API macros and functions not included in the glossary, refer to the Runner or Test API Reference document for information.

2. Glossary

–A–

- **API**: Application Programming Interface: public typedefs, structs, enums, macros, and functions that provide a well-defined service. For example, the BriteTest Runner API and the BriteTest Test API.
- **artifact**: A file or bundle of files produced by a GitHub Actions workflow and stored for later download (e.g., build outputs, logs, reports). See also test artifact.

–B–

- **Bash glob pattern**: A shell wildcard pattern used for filename and path matching in Bash. In BriteTest `genpdf`, this is used for relative-path filtering with `-i` and `-x`. Default matching rules: `*` and `**` match any characters (including `/`), `?` matches one character (including `/`), `[abc]` matches one character from a set or range, and any other character matches itself. This differs from common path-segment glob expectations where `*` does not match `/`. With the `-g` option, `genpdf` uses segment-style matching: `*` and `?` do not match `/`, while `**` matches across `/`.

–C–

- **category†**: A named set of `RA_GROUP` and `RA_TEST` macros whose combined results are written by `RA_WRITE_RESULT` to the report with a specified category name.
- **CD (Continuous Delivery/Deployment)**: Automated practices that extend CI (Continuous Integration) by packaging, releasing, or deploying software after it has passed all required tests. Continuous Delivery prepares release artifacts for manual approval, while Continuous Deployment automatically deploys every passing change to a target environment. Use continuous-delivery, continuous-deployment, or CI when used as an adjective.
- **CI (Continuous Integration)**: An automated practice where every change to a codebase is built and tested in a shared environment. A CI system runs workflows that compile the project, execute tests, validate formatting or static analysis rules, and produce artifacts such as logs or reports. CI helps detect errors early, ensures consistent build quality, and provides rapid feedback to developers. Use continuous-integration or CI when used as an adjective.
- **command line**: A line of text entered into a shell that specifies an executable and optional arguments or flags (options). Use command-line when used as an adjective.
- **concurrent block†**: A set of tests bracketed by `RA_BEGIN_CONCURRENT` and `RA_END_CONCURRENT` macros.
- **contributor†**: Any person or agent that may commit to BriteTest `main`. The implementation changes and documentation updates for a commit to `main` must conform to the BriteTest contribution guidelines and must have approval by a designated approver or agent.
- **control file**: See golden file.
- **customization function†**: A Runner API function that can be used to help customize the orchestrator (main) function and test group functions.

–D–

- **default report filename†**: The report filename used when only a directory path (or no `PATH`) is provided.

–E–

- **executable**: A compiled/linked program, for example, one that contains, for BriteTest, the orchestrator, test groups, test expressions (and underlying functions), and the BriteTest APIs. It is run from a shell using a command line.

–F–

- **fail†**: A counted failure where the test expression for an `RA_TEST` macro evaluates to zero.
- **fault†**: A counted test group or test fault (e.g., invalid memory access) captured by a BriteTest guard. See also fault type, guard, `RA_FAULT`, and isolation.
- **fault type†**: The various types of faults (e.g., `SIGSEGV`, `SIGBUS`, `SIGABRT`) that can occur or be injected. See also fault, `RA_FAULT` and `-I`.
- **framework†**: Guidelines, templates, APIs, tools, and documentation for a class of projects that simplify development within that class. For example, the BriteTest framework simplifies test development.

–G–

- **golden file†**: A previously generated file that can be compared to a newly generated file for differences. Differences (other than expected ones like timestamps) typically indicate a test failure. Sometimes the golden file is out of date and must be replaced by promoting the new file. See also output file.
- **group†**: See test group.
- **guard†**: The protection mechanism used to catch faults and continue test execution. See also fault, fault type, isolation, isolation mode, thread isolation, and process isolation.
- **guard level†**: The nesting depth of active guards for test groups and test expressions.

–H–

No terms currently defined.

–I–

- **`-I`†**: An optional command-line flag that enables an `RA_TEST` macro with an argument value of `I` to be executed. Typically used to inject a fail or fault into a run of a test executable. Default is to skip the `RA_TEST` macro if it has an argument value of `I`. See also fault, `RA_FAIL`, and `RA_FAULT`.
- **`-I<n>`†**: An optional command-line flag that enables an `RA_TEST` macro with an argument value of `<m>` between 1 and 9 to execute if `<m>` is between 1 and `<n>`. `<n>` must be between 1 and 9. Default for `<n>` is 9 if this flag is not specified and, for `<m>` is 1. If `<m>` is 0, the `RA_TEST` macro is not enabled (i.e., it is skipped). At most one `-I<n>` flag specified for a command line.

- **isolation**: A mechanism that prevents failures and faults in one component from affecting other components of an executable or system.
- **isolation mode†**: An execution mode for `RA_GROUP` and `RA_TEST` macros: `0` = same thread, `1` = separate thread, `2` = separate process.

–J–

- **job**: A unit of work within a workflow. A job runs a series of steps in a specified environment (such as a container or virtual machine). Jobs may run sequentially or in parallel, based on their dependencies.

–K–

No terms currently defined.

–L–

- **Runner Framework†**: Guidelines, templates, APIs, tools, and documentation for building and running test executables.
- **Runner API†**: An API that helps simplify implementing a test runner.
- **Test API†**: An API that helps simplify implementing tests.
- **`ra_internal_*`†**: A prefix for Runner API internal names. Do not define, declare or use names with this prefix.
- **`RA_INTERNAL_*`†**: A prefix for Runner API internal names. Do not define, declare or use names with this prefix.
- **`ra_*`†**: Prefix for Runner API functions, typedefs, structs, and v variable names. See the Runner API Reference.
- **`RA_*`†**: Prefix for Runner API macros and enum values. See the Runner API Reference.
- **`ta_internal_*`†**: A prefix for Test API internal names. Do not define, declare or use names with this prefix.
- **`TA_INTERNAL_*`†**: A prefix for Test API internal names. Do not define, declare or use names with this prefix.
- **`ta_*`†**: Prefix for Test API functions, typedefs, structs, and variable names. See the Test API Reference.
- **`TA_*`†**: Prefix for Test API macros and enum values. See the Test API Reference.
- **`RA_GROUP`†**: A Runner API macro that executes a test group function with a given isolation mode, maxparallel, and other parameters. See Runner API Reference.
- **`RA_TEST`†**: A Runner API macro that executes a test expression with a given isolation mode and other parameters. See the Runner API Reference.
- **`RA_FAIL`†**: A macro that returns 0. Typically used in the `RA_TEST` macro to conditionally inject a fail. See also `-I`.
- **`RA_FAULT`†**: A macro that injects (signals) a fault. Typically used in the `RA_TEST` macro to conditionally inject a fault. See also fault, `-I`, and the Runner API Reference.

–M–

- **`maxargs`†**: The maximum number of command-line arguments allowed by the `RA_PARSE_ARGS` macro. This macro parses only the first two arguments; additional

arguments require custom code.
- **`maxparallel`†**: The upper bound on concurrent `RA_GROUP` and `RA_TEST` macros. That is, when the number of macros executing equals `maxparallel`, the next macro to execute is delayed until one of the executing macros finishes. `maxparallel` is a parameter for the `RA_INIT_ORCHESTRATOR` and `RA_GROUP` macros.

–N–

- **`notes`†**: A string parameter for the `RA_CLOSE_REPORT` macro. This macro appends the string (which must include `\n` at the end of each line in the string) if the string is not NULL or empty. Alternatively or in addition, notes can be appended from a file containing notes.

–O–

- **orchestrator†**: See orchestrator (`main`) function.
- **orchestrator function†**: See orchestrator (`main`) function.
- **orchestrator (`main`) function†**: The `main` function of a runner executable (i.e., the test runner) that uses `RA_GROUP` macros to execute sets of tests (i.e., a test group) or `RA_TEST` macros to execute a specific test (i.e., a test expression).
- **output file†**: TODO. See also golden file.

–P–

- **pass†**: A counted success where the test expression for an `RA_TEST` macro evaluates to non-zero.
- **PATH†**: Optional command-line argument indicating the output destination; may be a report file path or directory path. Argument must be quoted if it contains spaces.
- **process guard†**: The guard used for process isolation. Unlike thread guards, a process guard can capture all signals but increases test execution time. For proven tests, use thread guards; otherwise, use process guards.
- **process isolation†**: An isolation mode where a test group or test expression runs in a separate process. See also isolation mode.
- **project†**: A single-token project identifier used in orchestrator initialization and default report naming.

–Q–

No terms currently defined.

–R–

- **report**: See test report.
- **report header†**: Lines of text written at the beginning of a test report that include the report title, a timestamp, etc.
- **runner†**: See test runner.
- **runner**: (GitHub) A machine or environment that executes the jobs defined in a GitHub Actions workflow. A runner provides the operating system, tools, and runtime needed

to perform workflow steps such as building, testing, or packaging a project. GitHub provides hosted runners, and users may also configure self-hosted runners.

–S–

- **semantic versioning**: A versioning scheme for artifacts. For example, for the Runner API or Test API, `M.m.p` (for `.h` and `.c` files) or `M.u` (document files) where M, m, p, and u are one or two digits and M indicates the major release, m indicates the minor release, p indicates the patch version, and u the document update version.
- **shell**: A command-line interface that allows a user or script to submit command lines. Examples include `pwsh`, `powershell.exe`, `bash`, `sh`, `zsh`, and `cmd.exe`.
- **step**: An individual action within a job. A step may run a shell command, execute a script, or invoke a reusable action. Steps run in order and share the job's execution environment.

–T–

- **test†**: See test expression.
- **test artifact†**: A specific kind of artifact, i.e., file or output generated by a runner executable (for example, a test report, `stdout`, and `stderr`).
- **test case**: This term is not used in the documentation. In other contexts, it may mean a single test or a set of tests; the documentation uses test expression for an individual test and test group for a set of test expressions.
- **test expression†**: An expression that is an argument of an `RA_TEST` macro that can be cast to `int`; zero means fail, non-zero means pass. A test expression and its underlying functions are user-written. The Test API is provided to help simplify writing a test expression and its underlying functions.
- **test group†**: A grouping of `RA_TEST` macros and optionally `RA_GROUP` macros.
- **test group function†**: A function declared with a `RA_DECLARE_GROUP` macro and followed by a function body in `{ }`.
- **test function†**: A user-written function used in implementing a test expression.
- **test helper function†**: A Test API function that helps simplify implementing a test expression.
- **test report†**: A file written by a test runner that records the results of a test run, including pass/fail counts, faults, and optional notes. See also report header, title, and notes.
- **test runner†**: An executable that runs a set of tests.
- **test suite**: A complete set of tests for a project or a subset of tests for a project. The documenation uses the term test group if it is a subset of the tests for a project. The documentaton does not use the term *test suite* since whether a set of tests is *complete* for a project is not well-defined.
- **testing artifact†**: See test artifact.
- **thread guard†**: The guard used for thread isolation. A thread guard can only reliably capture synchronous signals (`SIGSEGV`, `SIGBUS`, `SIGFPE`, and `SIGILL`). Other signals (`SIGABRT`, `SIGKILL`, `SIGSTOP`, `SIGTERM`, `SIGINT`, `SIGHUP`, `SIGQUIT`, `SIGPIPE`, `SIGALRM`, `SIGCHLD`, `SIGUSR1`, and `SIGUSR2`) cause the executable to terminate. A process guard can capture all signals but increases the time to run the tests. For already proven tests,

use thread guards; otherwise, use process guards.
- **thread isolation†**: An isolation mode where a test group or test expression executes in a separate thread. See also isolation mode.
- **title†**: An optional report header text provided when opening the report with an `RA_OPEN_REPORT` macro.

–U–

No terms currently defined.

–V–

No terms currently defined.

–W–

- **workflow**: A defined sequence of automated steps executed by a CI (continuous-integration) system. In GitHub Actions, a workflow is triggered by an event (such as a push, pull request, or scheduled run) and runs one or more jobs that perform tasks like building, testing, or packaging a project. Workflows may produce artifacts such as logs, reports, or build outputs.

–X–

No terms currently defined.

–Y–

No terms currently defined.

–Z–

No terms currently defined.